

Developer Guide

Vol. 8 -- Two Ways to Inject Dependencies

Topics covered in this volume:

- Hilt DI vs Manual DI -- a direct comparison
 - The Dependencies container and lazy initialisation
 - LocalDependenciesScope -- CompositionLocal for DI
 - ViewModelFactory helper function
 - Option A: inline viewModelFactory{} in the screen
 - Option B: companion object factory in the ViewModel
 - App.kt and MainActivity.kt wiring
 - Full side-by-side code comparison for every layer
 - Dart / Flutter equivalent patterns
-

1. The Two Approaches

This project demonstrates two ways to inject a repository into a ViewModel. Both produce identical runtime behaviour -- the only difference is who writes the wiring code: Hilt's annotation processor or you.

Hilt (previous approach)

```
// ViewModel -- Hilt reads the @Inject annotation and generates a factory
@HiltViewModel
class CartViewModel @Inject constructor(
    private val cartRepository: ICartRepository,
) : ViewModel()

// Screen -- hiltViewModel() uses the generated factory automatically
val viewModel: CartViewModel = hiltViewModel()
```

Manual DI (current approach)

```
// ViewModel -- plain constructor, no annotations
class CartViewModel(
    private val cartRepository: ICartRepository,
) : ViewModel()

// Screen -- you create the factory and pass the dependency explicitly
val container = LocalDependenciesScope.current
```

```
val viewModel: CartViewModel = viewModel(
    factory = viewModelFactory { CartViewModel(container.cartRepository) }
)
```

With Hilt: fewer lines in the screen, but Hilt processes annotations at compile time and generates invisible code. When something breaks, errors point to generated files.

With manual DI: one extra line per screen, but the full wiring is visible and debuggable. No annotation processing, faster builds, no framework to learn.

2. Dependencies.kt -- The Container

Dependencies is a plain Kotlin class that holds all singleton instances. It replaces Hilt's `@Module` + `@Binds` + `@Singleton` system entirely.

```
class Dependencies(context: Context) {

    // `by lazy` -- created on first access, then cached.
    // HILT EQUIVALENT: @Singleton (Hilt also defers creation to first use).
    val sharedPreferencesManager: SharedPreferencesManager by lazy {
        SharedPreferencesManager(context)
    }

    val cartRepository: ICartRepository by lazy {
        CartRepositoryImpl()
    }

    // Eager -- created immediately when Dependencies is constructed.
    val favoritesRepository: IFavoritesRepository = FavoritesRepositoryImpl()
    val productRepository: IProductRepository = ProductRepositoryImpl()

    // Dependencies that need another dependency pass it explicitly.
    // HILT EQUIVALENT: Hilt would inject SharedPreferencesManager
    // automatically via @Inject constructor.
    val orderHistoryRepository: IOrderHistoryRepository =
        OrderHistoryImpl(sharedPreferencesManager)
    val orderRepository: IOrderRepository =
        OrderRepositoryImpl(sharedPreferencesManager)
}
```

lazy vs eager initialisation

Both are valid. Use ``by lazy`` when the object is expensive to create or might not always be needed. Use eager (plain ``=``) for cheap, always-needed objects.

	by lazy { ... }	val x = SomeClass()
Created when?	First access	When Dependencies is constructed
Thread safe?	Yes (by default)	Yes
Good for?	Expensive or optional deps	Cheap, always-needed deps
Hilt equivalent	@Singleton (lazy by nature)	@Singleton (also lazy -- shown here as eager for contrast)

3. App.kt -- Building the Container

The Application class is created once when the process starts. It is the right place to build Dependencies because it lives as long as the app.

```
// BEFORE (Hilt)
@HiltAndroidApp          // annotation triggers Hilt's code generator
class App : Application()
// Hilt silently creates its SingletonComponent here and wires all @Singleton bindings.

// AFTER (manual DI)
class App : Application() {
    lateinit var container: Dependencies // non-null, assigned in onCreate()

    override fun onCreate() {
        super.onCreate()
        container = Dependencies(this) // `this` = Application context, safe to hold
    }
}
```

Dart equivalent: the main() function where you initialise GetIt or call WidgetsFlutterBinding.ensureInitialized() before runApp().

4. LocalDependenciesScope -- Passing the Container to Composables

Composable functions cannot use @Inject. CompositionLocal is Compose's built-in mechanism for passing values implicitly down the tree.

Definition (LocalDependenciesScope.kt)

```
// staticCompositionLocalOf = the entire subtree re-composes if value changes.
// Use this when the value NEVER changes (Dependencies is created once at startup).
val LocalDependenciesScope = staticCompositionLocalOf<Dependencies> {
    error("No AppContainer provided") // crash early if provider is missing
}
```

Providing it (MainActivity.kt)

```
// BEFORE (Hilt)
@AndroidEntryPoint      // tells Hilt to inject into this Activity
class MainActivity : ComponentActivity() {
    override fun onCreate(...) {
        setContent { AppNavigation() } // hiltViewModel() worked inside NavHost
    }
}

// AFTER (manual DI)
class MainActivity : ComponentActivity() {
    override fun onCreate(...) {
        enableEdgeToEdge()
        val container = (application as App).container
        setContent {
            OnlinestoreclaudeTheme {
                // Makes `container` accessible to every composable below
                CompositionLocalProvider(LocalDependenciesScope provides container) {

```

```

        AppNavigation()
    }
}
}
}
}

```

Reading it in any screen

```

@Composable
fun CartScreen(...) {
    // Reads the nearest LocalDependenciesScope from the tree
    val container = LocalDependenciesScope.current

    val viewModel: CartViewModel = viewModel(
        factory = viewModelFactory { CartViewModel(container.cartRepository) }
    )
}

```

Dart equivalent: using context.read() after wrapping the widget tree in Provider(create: { } => container, child: ...)

5. ViewModelFactory Helper (core/ViewModelFactory.kt)

ViewModelProvider.Factory is the interface Android uses to construct ViewModels with constructor parameters. Writing the full anonymous object is verbose, so a top-level helper function was added:

```

fun <VM : ViewModel> viewModelFactory(initializer: () -> VM): ViewModelProvider.Factory {
    return object : ViewModelProvider.Factory {
        override fun <T : ViewModel> create(modelClass: Class<T>): T {
            return initializer() as T
        }
    }
}

```

Without this helper, every screen would need to write:

```

val factory = object : ViewModelProvider.Factory {
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        return CartViewModel(container.cartRepository) as T
    }
}
val viewModel: CartViewModel = viewModel(factory = factory)

```

With the helper:

```

val viewModel: CartViewModel = viewModel(
    factory = viewModelFactory { CartViewModel(container.cartRepository) }
)

```

Jetpack also provides its own viewModelFactory { initializer { ... } } DSL (androidx.lifecycle.viewmodel.viewModelFactory). The custom helper above does the same thing with simpler syntax -- both are equivalent.

6. Option A vs Option B -- Two Factory Styles

Once you have the `viewModelFactory` helper and `LocalDependenciesScope`, there are two clean ways to create a `ViewModel` in a screen. Both are used in this project.

Option A -- inline factory in the screen (no companion object)

The factory lambda is written directly in the screen. The `ViewModel` stays clean.

```
// CartViewModel.kt -- no companion object, just a plain class
class CartViewModel(
    private val cartRepository: ICartRepository,
) : ViewModel() { ... }

// CartScreen.kt -- factory is inline
val container = LocalDependenciesScope.current
val viewModel: CartViewModel = viewModel(
    factory = viewModelFactory { CartViewModel(container.cartRepository) }
)
```

Used in: `CartScreen`, `OrderHistoryScreen`, `FavoritesScreen`, `OrderScreen`

Option B -- companion object factory in the ViewModel

The factory logic lives next to the `ViewModel`. The screen calls a named factory method.

```
// ProductsViewModel.kt -- factory in companion object
class ProductsViewModel(
    private val productRepository: IProductRepository,
) : ViewModel() {

    companion object {
        fun factory(productRepository: IProductRepository) = viewModelFactory {
            ProductsViewModel(productRepository)
        }
    }
}

// ProductsScreen.kt -- screen calls the named factory
val container = LocalDependenciesScope.current
val viewModel: ProductsViewModel =
    viewModel(factory = ProductsViewModel.factory(container.productRepository))
```

Used in: `ProductsScreen`, `ProductDetailScreen`

Comparison

	Option A (inline)	Option B (companion)
Factory location	In the screen	In the <code>ViewModel</code>
<code>ViewModel</code> cleanliness	No boilerplate	companion object added
Screen cleanliness	Factory lambda visible	One clean method call
Refactoring	Change in each screen	Change in one companion
Best for	Simple <code>ViewModels</code> used in 1 screen	<code>ViewModels</code> used in multiple screens

This project uses both intentionally so you can compare them side by side. In a real project, pick one style and be consistent.

7. Commented-out Companion Objects in ViewModels

Some ViewModels (CartViewModel, OrderHistoryViewModel) have the companion object factory commented out. This is intentional -- it shows both styles coexisting for learning purposes.

```
// CartViewModel.kt
class CartViewModel(
    private val cartRepository: ICartRepository,
) : ViewModel() {

    // OPTION B (currently unused -- screen uses Option A inline factory instead)
    //
    // companion object {
    //     fun factory(cartRepository: ICartRepository) = viewModelFactory {
    //         CartViewModel(cartRepository)
    //     }
    // }
    //
    // To switch to Option B:
    // 1. Uncomment this companion object
    // 2. In CartScreen, replace:
    //     viewModelFactory { CartViewModel(container.cartRepository) }
    // with:
    //     CartViewModel.factory(container.cartRepository)
}
```

The commented code is not dead code -- it is an educational alternative that shows the reader exactly how to switch between the two styles.

8. Why @AssistedInject Was Needed and Is Now Gone

Before the migration, OrderViewModel and OrderHistoryViewModel used @AssistedInject -- a Hilt feature for injecting dependencies that are only known at runtime (not at compile time).

The @AssistedInject approach (removed)

```
// OrderHistoryViewModel.kt (BEFORE)
@HiltViewModel(assistedFactory = OrderHistoryControllerFactory::class)
class OrderHistoryViewModel @AssistedInject constructor(
    @Assisted private val orderHistoryRepository: IOrderHistoryRepository,
) : ViewModel()

@AssistedFactory
interface OrderHistoryControllerFactory {
    fun create(repo: IOrderHistoryRepository): OrderHistoryViewModel
}

// OrderHistoryScreen.kt (BEFORE) -- needed EntryPointAccessors to escape Hilt
val repo = EntryPointAccessors
    .fromApplication(context.applicationContext, AppEntryPoint::class.java)
    .orderHistoryRepository()
val viewModel = hiltViewModel<OrderHistoryViewModel, OrderHistoryControllerFactory> {
    factory -> factory.create(repo)
}
```

The manual DI approach (now)

```
// OrderHistoryViewModel.kt (AFTER) -- plain class
class OrderHistoryViewModel(
    private val orderHistoryRepository: IOrderHistoryRepository,
) : ViewModel()

// OrderHistoryScreen.kt (AFTER) -- simple and direct
val container = LocalDependenciesScope.current
val viewModel: OrderHistoryViewModel = viewModel(
    factory = viewModelFactory { OrderHistoryViewModel(container.orderHistoryRepository) }
)
```

The entire `@AssistedInject` system -- `@AssistedFactory` interface, `AppEntryPoint`, `EntryPointAccessors` -- was replaced by two lines. Manual DI makes runtime dependency passing trivial because you are just calling a constructor.

9. Full Dependency Flow

Process starts

```
|
App.onCreate()
|
Dependencies(applicationContext)
|
+-- SharedPreferencesManager(context)           [by lazy]
+-- CartRepositoryImpl()                         [by lazy]
+-- FavoritesRepositoryImpl()                    [eager]
+-- ProductRepositoryImpl()                     [eager]
+-- OrderHistoryImpl(sharedPrefsManager)         [eager]
+-- OrderRepositoryImpl(sharedPrefsManager)      [eager]
```

```
MainActivity.setContent {
    CompositionLocalProvider(LocalDependenciesScope provides container) {
        |
        AppNavigation()
        |
        CartScreen() {
            val container = LocalDependenciesScope.current
            viewModel(factory = viewModelFactory {
                CartViewModel(container.cartRepository) // singleton from Dependencies
            })
        }
        |
        ProductsScreen() {
            val container = LocalDependenciesScope.current
            viewModel(factory = ProductsViewModel.factory(container.productRepository))
        }
        |
        OrderScreen() {
            val container = LocalDependenciesScope.current
            viewModel(factory = viewModelFactory {
                OrderViewModel(container.orderRepository, container.cartRepository)
            })
        }
    }
}
```

```
    }  
  }  
}
```

Every screen reads from LocalDependenciesScope. Every dependency is a singleton from Dependencies
-- no repository is ever constructed more than once.

End of Vol. 8